

Explicación del script de extracción de tweets

Filtrado por palabra clave y estimación de tweets de Argentina

¿Qué hace este script?

El script se conecta a la plataforma X/Twitter usando la librería `twscrape` y busca tweets que contengan una palabra clave específica en español y publicados entre ayer y hoy. Luego intenta estimar cuáles de esos tweets podrían ser de Argentina y guarda los resultados en un archivo CSV para analizarlos después.

Cómo define el período de búsqueda

Toma la fecha de hoy y calcula automáticamente la de ayer. A partir de eso arma las variables `SINCE` y `UNTIL`, que se usan en la consulta: se buscan tweets publicados entre ayer y hoy, sin necesidad de modificar fechas manualmente.

La búsqueda principal

La función `buscar_y_guardar` arma una query como: "palabra" lang:es since:AAAA-MM-DD until:AAAA-MM-DD. Con esa consulta se descargan hasta N tweets (según el límite elegido) que coincidan con la palabra buscada.

Cómo estima si un tweet es argentino

Se usa la función `es_argentino`, que aplica dos pasos: 1) Si el tweet tiene información de lugar y el `countryCode` es AR, se marca como argentino. 2) Si no hay lugar explícito, se analiza la ubicación del perfil del usuario. Se limpia el texto y se buscan palabras clave típicas de Argentina (por ejemplo Buenos Aires, Córdoba, Rosario, CABA, etc.). Si alguna coincide, se asume que el tweet probablemente es argentino.

Generación del CSV

Con los tweets estimados como argentinos se crea una tabla con `pandas` que incluye: id del tweet, fecha, usuario, ubicación del perfil y texto del tweet. Luego se guarda automáticamente en un archivo llamado: `tweets_palabra_fecha_ARG.csv`. Ese archivo puede abrirse en Excel, R, Python o Power BI para análisis posteriores.

Cómo se ejecuta

Al final del archivo está la función `main`, que define la palabra clave (por ejemplo "milei") y ejecuta todo el proceso. El programa corre cuando se ejecuta el archivo en Python y al terminar informa cuántos tweets totales y argentinos detectó.

Cómo agregué la cuenta en mi implementación

En lugar de usar un archivo externo `.txt` con cuentas, la cuenta se definió directamente dentro del propio script de Python. La lógica fue la siguiente: 1) Se importó la librería `twscrape`. 2) Se creó el objeto API que maneja la conexión. 3) Se agregó manualmente una cuenta al pool con usuario, contraseña y cookies. Esto se hizo usando instrucciones del estilo: - crear la API: `api = API()` - agregar la cuenta directamente en código: `await api.pool.add_account( username = 'usuario', password = '*****', email = 'correo',`

`email_password = '*****', cookies = 'auth_token=...; ct0=...' )` Es una forma más directa porque todo queda en un único archivo y no hace falta manejar archivos externos. Si en el futuro se quisiera hacer el proceso más general, se podría pasar a la versión con archivo .txt, pero para esta etapa del proyecto esta solución funciona correctamente.